

IoT & Wireless Communications

Master Radio Networks

Dr. Mohamed Kas

Contents

1	Part 1: LoRa Physical Layer - Chirp Spread Spectrum	2
1.1	Task 1.1: Simulating Uplink and Downlink CSS (IQ Inversion)	2
1.2	Task 1.2: Question on Spreading Factor Orthogonality	2
2	Part 2: LoRaWAN MAC Layer & Topology Visualization	3
2.1	Task 2.1: Network Topology Visualization	3
2.2	Task 2.2: Time-on-Air (ToA) Calculation	3
2.3	Task 2.3: Realistic ALOHA Simulation with Progress Bar	3
3	Part 3: Step-by-Step Smart City Simulation in CupCarbon	4
3.1	Task 3.1: Defining Nodes via GPS and Hardware Parameters	4
3.2	Task 3.2: Writing and Assigning SenScript	5
3.3	Task 3.3: Running the Simulation	6
4	Part 4: Comparative LPWAN Simulation	6
4.1	Task 4.1: Energy Analysis	6

1 Part 1: LoRa Physical Layer - Chirp Spread Spectrum

LoRa uses a proprietary modulation derived from Chirp Spread Spectrum (CSS). The symbol duration T_s depends strictly on the assigned Spreading Factor (SF) and the Bandwidth (BW):

$$T_s = \frac{2^{SF}}{BW} \quad (1)$$

1.1 Task 1.1: Simulating Uplink and Downlink CSS (IQ Inversion)

LoRaWAN separates uplink and downlink traffic at the physical layer to avoid gateway-to-gateway and node-to-node interference. It achieves this via **IQ Inversion**: uplinks use *up-chirps*, while downlinks use *down-chirps*. Data is encoded by cyclically shifting the starting frequency of these chirps.

Your Task: Write a Python script based on Algorithm 1 to generate the continuous time-domain signal for both an uplink and a downlink message using a complete 3-symbol payload (e.g., $S = [20, 85, 110]$). **Use the same Spreading Factor (SF = 7) for both signals.** Plot their continuous time-domain waveforms (as sinusoidal signals) to visually observe the frequency modulation, the cyclic wraps, and the phase continuity between the symbols.

Algorithm 1 Generate Shifted Payload Symbol

Require: Symbol S , SF, BW, f_s , boolean is_uplink

```

1:  $T_s \leftarrow 2^{SF}/BW$ 
2:  $f_{offset} \leftarrow (S/2^{SF}) \times BW$ 
3:  $t \leftarrow$  array from 0 to  $T_s$  with step  $1/f_s$ 
4: if  $is\_uplink$  then
5:    $f_{inst} \leftarrow (f_{offset} + \frac{BW}{T_s} \times t) \pmod{BW}$  ▷ Cyclic Up-chirp
6: else
7:    $f_{inst} \leftarrow (f_{offset} - \frac{BW}{T_s} \times t) \pmod{BW}$  ▷ Cyclic Down-chirp
8: end if
9:  $phase \leftarrow 2\pi \int f_{inst} dt$ 
10: return  $\cos(phase)$ 
```

Guidance:

- Generate each symbol individually using a loop, then combine the arrays end-to-end using `numpy.concatenate([list_of_arrays])` to form the complete payload signal.
- Do not use a spectrogram. Use standard `matplotlib.pyplot.plot(signal)` to plot the waveform as a sine wave. At $f_s = 250$ kHz and SF=7, a 3-symbol payload will only be around 768 samples long, making the individual sinusoidal waves clearly visible!

1.2 Task 1.2: Question on Spreading Factor Orthogonality

LoRaWAN networks allow multiple IoT devices to transmit to a single gateway on the exact same frequency channel at the exact same time without colliding.

Question: How is this technically possible at the physical layer? Prove your explanation by writing a Python script to generate a base up-chirp at SF7 and a base up-chirp at SF8. Compute and plot their cross-correlation.

Guidance:

- **Array Lengths:** Because T_s is longer for SF8 than SF7, the generated numpy arrays will be different sizes. To compute the cross-correlation, you must pad the shorter array (SF7) with zeros to match the length of the SF8 array. Use `numpy.pad(array, (0, padding_length), 'constant')`.
- **Correlation:** Use `scipy.signal.correlate(sig1, sig2, mode='full')`.

2 Part 2: LoRaWAN MAC Layer & Topology Visualization

2.1 Task 2.1: Network Topology Visualization

Before simulating network traffic, we need to build the physical topology. In a real LoRaWAN network, devices further from the gateway must use higher Spreading Factors (SF) to ensure their signal reaches the destination, compensating for path loss.

Your Task: 1. Generate the random X and Y coordinates for 100 IoT nodes uniformly distributed around a Gateway placed at (0,0) with a maximum radius of 5000 meters. 2. Assign an SF to each node based on its distance from the gateway (e.g., $d \leq 1000m \rightarrow SF7$, $d \leq 2000m \rightarrow SF8$, up to $SF12$). 3. Plot the network topology using `matplotlib.pyplot.scatter`, coloring the nodes based on their assigned SF.

Guidance:

To generate points uniformly inside a circle, generate a random angle θ between 0 and 2π , and a random radius $r = R_{max} \times \sqrt{U}$, where U is a random value between 0 and 1. Then compute $x = r \cos(\theta)$ and $y = r \sin(\theta)$.

2.2 Task 2.2: Time-on-Air (ToA) Calculation

Write a function that calculates the exact Time-on-Air (in milliseconds) based on payload size, SF, BW, and Coding Rate (CR). Test it with a 20-byte payload at SF10, BW 125 kHz, and CR 4/5.

Guidance:

Let PL be Payload size in bytes, $H = 1$ (explicit header), $DE = 0$ (data rate optimization disabled), and $CR = 1$ (for a 4/5 coding rate).

1. **Symbol Time:** $T_s = \frac{2^{SF}}{BW}$
2. **Preamble Time:** $T_{preamble} = (8 + 4.25) \times T_s$
3. **Number of Payload Symbols:**

$$n_{payload} = 8 + \max \left(\left\lceil \frac{8PL - 4SF + 28 + 16 - 20H}{4(SF - 2DE)} \right\rceil \times (CR + 4), 0 \right)$$
4. **Total ToA:** $ToA = T_{preamble} + (n_{payload} \times T_s)$

2.3 Task 2.3: Realistic ALOHA Simulation with Progress Bar

Using the `simpy` library, simulate the 100 nodes you generated in Task 2.1 for **1 virtual hour** (3,600,000 ms). To mimic a realistic IoT network, each node should transmit asynchronously to the gateway on average every **2 minutes** (120,000 ms).

Requirement: To understand the simulation's progress, integrate the `tqdm` library to display a progress bar in your console. Calculate the final Packet Delivery Ratio (PDR).

Guidance:

- **Node Process:** In SimPy, nodes must yield time between transmissions. Use:
`yield env.timeout(random.expovariate(1.0 / 120000))` to simulate the 2-minute interval.
- **Tracking Channels:** Initialize a dictionary in your Gateway class to track collision times independently per SF: `self.channel_active_until = {sf: 0 for sf in range(7, 13)}`.
- **Progress Bar (tqdm):** Step through the 1-hour simulation in 1-minute blocks to update the progress bar:
`for t in tqdm(range(0, 3600000, 60000)): env.run(until=t+60000)`

3 Part 3: Step-by-Step Smart City Simulation in CupCarbon

While Python is excellent for mathematical modeling, real IoT deployments require geographical context and strict hardware parameters. **CupCarbon** is a Smart City WSN simulator that maps nodes onto OpenStreetMap data.

Important Note: In CupCarbon, *all* devices are placed on the map as generic "Sensor Nodes". A node only becomes a "Gateway" or a "Sensing End-Node" based on how we configure its hardware parameters and the SenScript we assign to it.

3.1 Task 3.1: Defining Nodes via GPS and Hardware Parameters

To ensure all students simulate the exact same urban conditions, we will place our nodes using precise GPS coordinates around the Besançon region, and configure their exact hardware profiles.

1. Open CupCarbon (**File** → **New Project**). Go to **Map** → **Background** to load the map view. Center your map around the coordinates provided in Table 1.
2. **Adding Devices:** Look at the left toolbar. Click the **Sensor Node** icon (it looks like a small dot with radio waves) and click anywhere on the map to place a total of **6 generic nodes**.
3. **Defining the Gateway:** Right-click your first node and select **Parameters**.
 - Go to the **Location** tab and enter the Gateway coordinates (Table 1).
 - (*Optional*) To visually distinguish it, you can change its icon to a Sink/Base Station in the parameters.
4. **Defining the Sensing Nodes & Hardware:** Right-click the remaining 5 nodes to open their **Parameters**. You must configure the following for **each** sensing node to simulate real IoT hardware:
 - **Location:** Enter their precise GPS coordinates from Table 1.
 - **Radio Radius:** Adjust this value to mimic the assigned Spreading Factor (e.g., set Node 5's radius to 5000m).
 - **Coverage Direction & Angle:** Keep the angle at **360** to simulate an omnidirectional LoRa antenna.
 - **Energy Max:** Set to **10000 Joules** (Simulates initial battery capacity).

- **Sensing Cons:** Set to **0.05** (Energy consumed capturing data before transmission).
- **Drift:** Set to **0.5** (Simulates measurement noise/error inherent to cheap sensors).

Device Role	Latitude	Longitude	Radio Radius (Simulated SF)
Gateway	47.2431	5.9886	N/A (Receiving Mode)
Sensing Node 1	47.2450	5.9890	1000m (SF7)
Sensing Node 2	47.2480	5.9800	2000m (SF8)
Sensing Node 3	47.2550	5.9990	3000m (SF9)
Sensing Node 4	47.2300	6.0000	3800m (SF10)
Sensing Node 5	47.2700	6.0200	5000m (SF12)

Table 1: Precise GPS Coordinates for CupCarbon LoRaWAN Topology

3.2 Task 3.2: Writing and Assigning SenScript

We will now program the actual behavior of our devices.

1. In the top menu, click **Scripts** → **New Script**.
2. Paste the **Sensing End-Node Script** (below) into the editor. Save it as `sensor_tx.csc`. This script continuously sets a dummy temperature value and broadcasts it.
3. Create another new script for the **Gateway Script** (below). Save it as `gateway_rx.csc`. This script instructs the device to halt and listen (`wait`), read incoming payloads, and visually blink (`mark`) when successful.
4. **Assigning Scripts (CRITICAL STEP):** In the main map view, select all 5 of your Sensing Nodes (hold Shift or drag a box). In the script editor for `sensor_tx.csc`, click **Assign to selected node(s)**. Then, select *only* your Gateway node, and assign the `gateway_rx.csc` script to it.

Sensing End-Node Script (`sensor_tx.csc`):

```
1 loop
2     delay 5000          // Wait 5 seconds
3     set temp 22         // Simulate a sensor reading
4     send $temp *        // Broadcast the reading
5 endloop
```

Gateway Script (`gateway_rx.csc`):

```
1 loop
2     wait                // Listen for incoming RF packets
3     read payload        // Process the received data
4     mark                // Visually highlight the gateway in the GUI
5     delay 100
6     unmark              // Remove the highlight
7 endloop
```

3.3 Task 3.3: Running the Simulation

1. On the main toolbar, click the **Run Simulation** (Play) button.
2. You will see arrows shooting from the sensing nodes to the gateway. The gateway node will flash visually when it successfully reads a payload.
3. **Observation:** Stop the simulation. Reduce Sensing Node 5's Radio Radius to 1000m (simulating a drop from SF12 to SF7). Run it again. Does the message still arrive?

4 Part 4: Comparative LPWAN Simulation

Technology	Urban Range	Tx Interval (Simulated)	Energy Profile
LoRaWAN (SF10)	3 - 5 km	20 seconds	Low
Sigfox	10 - 15 km	600 seconds	Ultra-Low
NB-IoT (3GPP)	1 - 2 km (Cell tower)	2 seconds	Medium-High

Table 2: Simulated LPWAN Parameters

4.1 Task 4.1: Energy Analysis

Create three separate gateways (LoRaWAN, Sigfox, NB-IoT) and assign them different SenScript delays corresponding to Table 2. Ensure the **Energy Max** and **Sensing Cons** parameters are configured. Enable the **Battery Module** in CupCarbon, run the simulation, and compare the battery depletion slopes across the three networks. Based on your results, which technology would you choose for real-time vehicle tracking versus a rural soil moisture sensor?

Guidance:

- **Viewing the Graphs:** Stop the simulation after letting it run for a while. Go to the top menu *Results* → *Energy Consumption* to view the depletion graph.